

Acceptance test report

BCIS Subscription Billing and Collection System

Initial laboratory verification on 5 October 2026; final regression on 9 October 2026 (Asia/Manila).

Automated domain tests

9 passed: exact, partial, advance, oldest-first arrears, decimal parsing, allocation conservation, aging boundaries, role permission boundaries, and Manila calendar-day report windows.

PostgreSQL and authenticated API tests

18 passed against an isolated PostgreSQL 18 database:

- AT-01: PHP 999 exact payment creates receipt, zero invoice balance, PAID state, and balanced ledger.
- AT-02: PHP 500 on PHP 999 leaves PHP 499, PARTIALLY_PAID.
- AT-03: PHP 3,000 on PHP 1,000 retains credit; future invoice consumes credit without losing value.
- AT-04: PHP 1,200 against two PHP 999 invoices clears August and leaves September PHP 798.
- AT-05: duplicate GCash reference is rejected by a database uniqueness constraint. Direct unverified GCash posting is also rejected.
- AT-06: reversal preserves the original payment, restores invoice balance and ledger, and creates an audit entry.
- AT-07: PHP 20,000 collection and remittance yields zero difference.
- AT-08: PHP 20,000 collection and PHP 19,500 remittance yields PHP 500 shortage, retained in REMITTED state.
- AT-09: three concurrent payments on the same subscriber yield unique receipts and exactly the correct allocation; no over-allocation.
- AT-10: direct cashier requests to users and backups receive HTTP 403.
- AT-11: repeated monthly billing produces zero duplicate invoices.
- Additional: same payment idempotency key returns the original receipt.
- Additional: injected transaction failure leaves no partial payment behind.

Additional regressions passed for unchanged PostgreSQL calendar dates, immutable invoice voiding, blocking credits below allocated payments, and preserving finalized invoices after a future-rate change.

Backup and restore

AT-12 passed: the synthetic database was backed up, checksum verified, and restored transactionally into a different empty PostgreSQL database. All 50 subscribers and invoice/payment/allocation counts matched. Foreign keys were restored, no negative invoice balances were found, and sessions were invalidated. Evidence: backup-restore-evidence.json. A second drill verified attachment checksums and that a change made after the backup was absent from the restored snapshot. This was a restore drill, not an overwrite of the source database.

End-to-end workflow checks

The synthetic GCash proof was uploaded, reviewed, and atomically posted; the resulting payment appeared in the subscriber history. A field payment was assigned to a collection batch; PHP 500 shortage remained after explicit reconciliation. Evidence: workflow-evidence.json.

Browser checks cover login, navigation, subscriber ledger, and payment allocation preview. Screenshots are saved in tests/screenshots. Final UI test result is recorded in ui-evidence.json.

Build checks

Strict TypeScript check and production renderer/Electron bundle build pass. Actual Electron on macOS also passed secure login, sandbox/context isolation, no renderer Node access, binary GCash proof preview and native PDF report saving. Evidence: desktop-evidence.json. The actual Windows NSIS installation, Windows printing and three physical Windows client deployment remain environment-dependent release checks. Concurrent PostgreSQL/API testing does not substitute for physical LAN deployment.

Not yet established by testing

No benchmark has established performance at 20,000 subscribers / 500,000 invoices / 500,000 payments / 1,000,000 ledger entries. Queries and indexes support paginated operational screens, but profile ledger and exports still need large-dataset streaming/pagination before a production-scale performance claim. No external payment provider or Facebook integration is implemented; GCash verification is manual as specified.